



Facultad de Ciencias e Ingenierías
ESCUELA DE INGENIERÍA DE SOFTWARE

Asignatura: ISW-123-1
Asignación: Actividad 1. Herencia de Clases
Estudiante: Frankelly Cordero
Matrícula: 2024-3153
Fecha Límite: martes, 4 de agosto de 2026, 18:00

Aquí va el reporte, intentando ser conciso como lo pediste, pero sin dejar de lado la explicación necesaria para la asignatura.

Asignatura: ISW-123-1 **Fecha de Vencimiento:** martes, 4 de agosto de 2026, 18:00

Título: Actividad 1. Herencia de Clases

Introducción

En el desarrollo de software moderno, especialmente con lenguajes orientados a objetos como C#, la herencia es una piedra angular que permite construir sistemas más modulares y reutilizables. Nos ayuda a modelar relaciones del mundo real donde ciertas entidades comparten características y comportamientos, pero también tienen sus propias particularidades. Me parece que entender bien este concepto es fundamental para escribir código limpio y escalable.

¿Qué es herencia jerárquica en lenguaje de programación C#?

Cuando hablamos de herencia jerárquica en C#, nos referimos a un modelo donde una única clase base sirve como punto de partida para que múltiples clases derivadas hereden de ella. Piensa en esto como un árbol genealógico donde un padre tiene varios hijos, y cada hijo es distinto, pero todos comparten el mismo progenitor. La idea principal es centralizar la funcionalidad común en esa clase base, evitando así la duplicación de código en las clases especializadas.

Por ejemplo, imaginemos una clase `Vehiculo` que define propiedades como `Marca` y `Modelo`, y un método `Arrancar()`. De esta clase `Vehiculo` podríamos derivar `Coche`, `Moto` y `Camion`. Cada una de estas clases derivadas heredaría `Marca`, `Modelo` y `Arrancar()`, pero también podrían añadir sus propias propiedades únicas, como `NumeroPuertas` para `Coche` o `CapacidadCarga` para `Camion`. Esto simplifica mucho el mantenimiento y la extensión del código.

```

// Clase base
public class Vehiculo
{
    public string Marca { get; set; }
    public string Modelo { get; set; }

    public void Arrancar()
    {
        Console.WriteLine($"El {Marca} {Modelo} está arrancando.");
    }
}

// Clase derivada 1
public class Coche : Vehiculo
{
    public int NumeroPuertas { get; set; }

    public void AbrirMaletero()
    {
        Console.WriteLine($"El coche {Marca} {Modelo} abre su maletero.");
    }
}

// Clase derivada 2
public class Moto : Vehiculo
{
    public bool TieneCarenado { get; set; }

    public void Inclinar()
    {
        Console.WriteLine($"La moto {Marca} {Modelo} se inclina en la curva.");
    }
}

// Ejemplo de uso
public class Programa
{
    public static void Main(string[] args)
    {
        Coche miCoche = new Coche { Marca = "Toyota", Modelo = "Corolla", Numero
miCoche.Arrancar(); // Método heredado
miCoche.AbrirMaletero(); // Método propio

        Moto miMoto = new Moto { Marca = "Yamaha", Modelo = "MT-07", TieneCarena
miMoto.Arrancar(); // Método heredado
miMoto.Inclinar(); // Método propio
    }
}

```

```
}  
}
```

Características de la herencia en C

La herencia en C# viene con algunas particularidades importantes que hay que tener en cuenta. Primero, y esto es crucial, C# solo soporta herencia simple de clases. Esto significa que una clase puede heredar directamente de *una única* clase base. No puedes tener una clase que herede de dos o más clases al mismo tiempo, como sí permiten otros lenguajes. Esto, desde mi punto de vista, ayuda a mantener la claridad en la jerarquía y evita problemas de ambigüedad que a veces surgen con la herencia múltiple.

Otra característica fundamental es el uso de las palabras clave `virtual` y `override`. Con `virtual` en la clase base, indicamos que un método puede ser sobrescrito por una clase derivada. Luego, en la clase derivada, usamos `override` para proporcionar una implementación específica para ese método. Esto es la base del polimorfismo, permitiendo que objetos de diferentes tipos respondan de manera distinta a la misma llamada de método. Además, los modificadores de acceso como `protected` son súper útiles, ya que permiten que los miembros sean accesibles solo dentro de la clase y sus clases derivadas, manteniendo un buen encapsulamiento.

Tipos de herencia en programación orientada a objetos

Aunque C# tiene sus propias reglas, la programación orientada a objetos define varios tipos de herencia que son conceptualmente importantes. La **herencia simple** es la más básica, donde una clase hereda de una sola clase base; esta es la que C# implementa directamente. Luego tenemos la **herencia multinivel**, que ocurre cuando una clase hereda de otra, y a su vez, esta segunda clase es heredada por una tercera. Es como tener un abuelo, un padre y un hijo.

Está también la **herencia jerárquica**, que es la que ya explicamos: una clase base de la que heredan múltiples clases derivadas. Finalmente, existe la **herencia múltiple**, donde una clase intentaría heredar de dos o más clases base directamente. C# no permite esto para clases, pero logra un efecto similar a través de las interfaces, donde una clase puede implementar múltiples interfaces, obteniendo así comportamientos de varias "fuentes" sin los problemas de ambigüedad de la herencia múltiple de clases. La **herencia híbrida** es una combinación de dos o más de los tipos anteriores, aunque no se implementa directamente con clases en C#.
